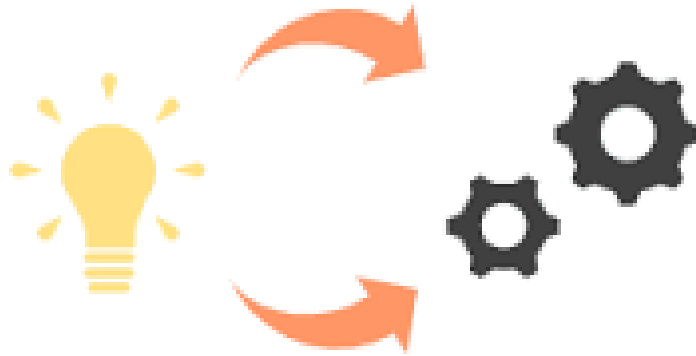


Implementatie- & Deploymentplan – UNASAT AI Student Mentor



Implementatie

Studenten: Nawiesh Ragho SE/1123/064

Aditije Matai SE/1123/043

Jozua Mertowirijo SE/1123/045

Reshwan Moenna SE/1123/047

Misael Karwofodi SE/1123/033

Opleiding: Software Engineering

Programmaonderdelen: Ontwikkel en deploy een productie klare chatbot

Docenten: Rwynn Christian

Debora Bergwijn

Datum: 12 Maart 2026

Inhoudsopgave

Inleiding.....	3
1. Vereisten	4
1.1 Hardware	4
1.2 Software.....	4
1.3 Externe accounts.....	4
1.4 Kennis.....	4
2. Stappenplan voor lokale deployment.....	4
2.1 Repository clonen	4
2.2 Configuratiebestand aanmaken.....	4
2.3 Controleer Docker en Docker Compose	5
2.4 Start de containers.....	5
2.5 Controleer of alles draait	5
2.6 (Optioneel) Achtergrondmodus.....	6
2.7 Stoppen van de containers	6
3. Configuratiebeheer	6
4. Versiebeheerstrategie.....	6
5. Updateprocedure.....	6
6. Rollbackstrategie.....	7
6.1 Via Git-tag	7
6.2 Zonder tag (via lokale image).....	7
6.3 Database rollback.....	8
7. Beheer en verantwoordelijkheden	8
8. Kosteninschatting.....	8
9. Probleemoplossing (Troubleshooting).....	9
9.1 Port already in use	9
9.2 Database verbindingfout	9
9.3 Groq API error (ongeldige key)	9
9.4 JWT decode error.....	9
9.5 Database init problemen	9
Bijlage.....	10

Inleiding

Doel van dit document

Dit implementatie- en deploymentplan beschrijft **hoe de UNASAT AI Student Mentor in gebruik wordt genomen**. Het is een praktisch stappenplan dat een IT-medewerker stap voor stap kan volgen om het systeem te installeren, configureren en beheren.

Geen theorie, alleen actie – ieder hoofdstuk bevat direct uitvoerbare commando's en instructies.

Wat staat er in dit plan?

1. **Vereisten** – Wat je nodig hebt voordat je begint
2. **Lokale deployment** – Stapsgewijze installatie
3. **Configuratiebeheer** – Hoe om te gaan met instellingen
4. **Versiebeheerstrategie** – Git workflow
5. **Updateprocedure** – Nieuwe versies uitrollen
6. **Rollbackstrategie** – Terugdraaien bij problemen
7. **Beheer & verantwoordelijkheden** – Wie doet wat
8. **Kosteninschatting** – Maandelijkse lasten
9. **Probleemoplossing** – Veelvoorkomende fouten en oplossingen

1. Vereisten

Voordat het systeem kan worden geïmplementeerd, moeten de volgende zaken aanwezig zijn:

1.1 Hardware

- Een machine (fysiek of virtueel) met minimaal **2 CPU cores** en **4 GB RAM** (aanbevolen 4 cores en 8 GB RAM voor productie).
- Vrije schijfruimte: minimaal **10 GB** (voor Docker-images, volumes, en logs).

1.2 Software

- **Besturingssysteem:** Linux (Ubuntu 20.04 of hoger aanbevolen), Windows (met Docker Desktop), of macOS.
- **Git** – voor het clonen van de repository.
- **Docker Engine** (versie 20.10+) en **Docker Compose** (versie 2.x+).
 - Installatiehandleidingen: [Docker install](#)
- **Internetverbinding** – voor het downloaden van Docker-images, Python-pakketten en voor de Groq API.

1.3 Externe accounts

- Een **Groq API-key** (gratis aan te vragen op console.groq.com).
- Optioneel: een domeinnaam (als de applicatie publiek toegankelijk moet zijn).

1.4 Kennis

- Basiskennis van de terminal, Docker, en omgevingsvariabelen.

2. Stappenplan voor lokale deployment

Volg onderstaande stappen om het systeem op een lokale machine of server te implementeren.

2.1 Repository clonen

Open een terminal en voer uit:

```
bash
```

```
git clone <repository-url>
```

```
cd UNASAT_CHATBOT
```

Vervang <repository-url> door de daadwerkelijke URL (bijv. van GitHub).

2.2 Configuratiebestand aanmaken

Maak een bestand .env in de hoofdmap en vul de volgende variabelen in:

ini

GROQ_API_KEY=gsk_jouw_groq_api_key

JWT_SECRET=een_zeer_lang_geheim_wachtwoord_van_minstens_32_tekens

JWT_ALGORITHM=HS256

- GROQ_API_KEY: verkrijgbaar via de Groq console.
- JWT_SECRET: kies een willekeurige, lange string (minimaal 32 tekens). Gebruik bijv. een wachtwoordgenerator.

2.3 Controleer Docker en Docker Compose

Controleer of Docker correct is geïnstalleerd:

bash

docker --version

docker-compose --version

Beide commando's moeten een versienummer tonen zonder foutmeldingen.

2.4 Start de containers

Voer het volgende commando uit in de projectmap:

bash

docker-compose up --build

- --build zorgt dat de Docker-image voor de app opnieuw wordt gebouwd (bij de eerste keer of na wijzigingen).
- Bij een volgende start zonder wijzigingen kun je docker-compose up gebruiken (zonder --build).

De eerste keer kan het enkele minuten duren omdat Docker de benodigde images (Python, MySQL) downloadt en de Python-pakketten installeert.

2.5 Controleer of alles draait

Na het starten zie je logs van beide containers. Open een browser en ga naar:

text

http://localhost:8000

De loginpagina van de studentenportal verschijnt. Log in met de testgebruiker:

- **Gebruikersnaam:** student1
- **Wachtwoord:** unasat123

Deze gebruiker is automatisch aangemaakt bij de eerste database-initialisatie.

2.6 (Optioneel) Achtergrondmodus

Om de containers op de achtergrond te laten draaien, voeg -d toe:

```
bash
```

```
docker-compose up -d
```

2.7 Stoppen van de containers

Druk Ctrl+C in de terminal waar de containers op de voorgrond draaien, of voer in een andere terminal uit:

```
bash
```

```
docker-compose down
```

Dit stopt en verwijdert de containers, maar laat het databasevolume intact (data blijft bewaard).

3. Configuratiebeheer

- **Omgevingsvariabelen:** Alle gevoelige instellingen (API-key, JWT secret) worden beheerd via het .env bestand. Dit bestand wordt **niet** opgenomen in versiebeheer (zie .gitignore).
- **Databaseconfiguratie:** De database-instellingen (gebruikersnaam, wachtwoord, databasenaam) staan in docker-compose.yml. In productieomgevingen kunnen deze worden overschreven via environment variabelen in de compose-file of via een apart .env bestand.
- **Statische bestanden:** De frontend-bestanden (index.html, admin.html, logo.png) worden via een volume gemount (- ./static:/code/static). Dit maakt het mogelijk om ze aan te passen zonder de container opnieuw te bouwen.

4. Versiebeheerstrategie

- **Git-repository:** Gebruik Git met feature branches. De hoofd branch heet main (of master).
- **Tags:** Bij elke stabiele release wordt een tag aangemaakt (bijv. v1.0.0). Dit maakt het eenvoudig om terug te gaan naar een eerdere versie.
- **Commitberichten:** Gebruik duidelijke, beschrijvende commitberichten (bijv. "Added rate limiting to login endpoint").

5. Updateprocedure

Wanneer er een nieuwe versie van de applicatie beschikbaar is, volg dan deze stappen:

1. Wijzigingen ophalen:

```
bash
```

```
git pull origin main
```

2. **Controleer of er wijzigingen in de .env nodig zijn** (bijv. nieuwe variabelen). Pas zo nodig aan.

3. **Herbouw en herstart de containers:**

```
bash
```

```
docker-compose down
```

```
docker-compose up --build -d
```

De --build zorgt dat de nieuwe code wordt gebruikt. De -d start de containers op de achtergrond.

4. **Test de applicatie** door de belangrijkste functionaliteiten te controleren (inloggen, chat, admin-pagina).

Let op: Tijdens de herstart is de applicatie korte tijd niet beschikbaar. Dit kan worden geminimaliseerd door eerst de nieuwe containers te bouwen en daarna pas de oude te stoppen (bijv. met docker-compose up --build --no-start gevolgd door docker-compose restart). Voor eenvoud wordt bovenstaande procedure aanbevolen.

6. Rollbackstrategie

Mocht een update mislukken of ongewenste effecten hebben, dan kan worden teruggegaan naar de vorige stabiele versie.

6.1 Via Git-tag

Als er een tag is gemaakt van de vorige versie:

```
bash
```

```
git checkout v1.0.0 # of de gewenste tag
```

```
docker-compose down
```

```
docker-compose up --build -d
```

6.2 Zonder tag (via lokale image)

Als er geen tag is, maar de vorige image nog op de machine staat:

```
bash
```

```
docker images
```

Zoek de image van de vorige versie (bijv. unasat_chatbot-app:latest). Je kunt de compose-file aanpassen om een specifieke image-tag te gebruiken, of simpelweg de containers opnieuw starten met de bestaande image (geen --build):

```
bash
```

```
docker-compose down
```

```
docker-compose up -d # zonder --build, gebruikt de bestaande image
```

6.3 Database rollback

Als de update ook databasewijzigingen bevatte, kan het nodig zijn om de database terug te zetten naar een backup. Zie paragraaf 9 voor back-upstrategie.

7. Beheer en verantwoordelijkheden

Rol	Verantwoordelijkheden
Ontwikkelaar	– Schrijven en testen van code. – Documenteren van wijzigingen. – Aanmaken van releases/tags.
Beheerder	– Uitvoeren van deployments volgens dit plan. – Bewaken van logs en statistieken (via admin-pagina). – Beheren van de .env en API-keys. – Monitoren van schijfruimte en prestaties.
Helpdesk / ondersteuning	– Eerste lijn bij gebruikersvragen. – Signaleren van terugkerende problemen aan de beheerder.

8. Kosteninschatting

Het systeem is ontworpen om met minimale kosten te draaien, met name door gebruik van gratis tiers.

Onderdeel	Geschatte kosten per maand	Opmerkingen
Groq API	€0,-	Gratis tier: 14.400 requests/maand. Bij overschrijding kan worden overgestapt op betaald (prijzen op aanvraag).
Hosting (VPS)	Vanaf €5,-	Bijv. DigitalOcean droplet (2 GB RAM, 1 vCPU) of een goedkopere provider.
Domeinnaam	€10,- per jaar (optioneel)	Indien gewenst voor publieke toegang.

Onderdeel	Geschatte kosten per maand	Opmerkingen
Totaal (eerste jaar)	Circa €70,- (incl. domein)	Na het eerste jaar alleen hostingkosten.

Besparingstips:

- Gebruik een goedkope VPS zoals Contabo (vanaf €4,50/maand).
- Overweeg om de applicatie tijdelijk op een eigen server te draaien zonder domein (alleen lokaal netwerk).
- Houd het verbruik van de Groq API in de gaten; 14.400 requests is voor een school met 100 studenten (elk 10 chats/maand) meer dan genoeg.

9. Probleemoplossing (Troubleshooting)

9.1 Port already in use

Foutmelding: Error starting userland proxy: listen tcp4 0.0.0.0:8000: bind: address already in use

Oplossing: Een andere applicatie gebruikt poort 8000. Stop die applicatie, of wijzig de poort in docker-compose.yml (bijv. "8001:8000").

9.2 Database verbindingsfout

Foutmelding: Can't connect to MySQL server on 'db' (113)

Oplossing: De database-container is mogelijk nog niet klaar. Wacht even en probeer opnieuw. Controleer met docker-compose logs db of de database gezond is.

9.3 Groq API error (ongeldige key)

Foutmelding: {"error": "Invalid API Key"}

Oplossing: Controleer de GROQ_API_KEY in .env. Zorg dat er geen spaties of aanhalingstekens omheen staan.

9.4 JWT decode error

Foutmelding: JWTError in logs.

Oplossing: Controleer of JWT_SECRET in .env consistent is en lang genoeg. Herstart de containers.

9.5 Database init problemen

Als de database niet correct wordt geïnitieerd (bijv. tabellen ontbreken), kan het nodig zijn het volume te wissen en opnieuw te beginnen:

```
bash
```

```
docker-compose down -v # verwijdert ook volumes (data kwijt!)
```

```
docker-compose up --build
```

Bijlage – Handige commando's

Commando	Doel
<code>docker-compose up</code>	Start containers op de voorgrond
<code>docker-compose up -d</code>	Start containers op de achtergrond
<code>docker-compose down</code>	Stop en verwijder containers
<code>docker-compose logs -f</code>	Volg alle logs real-time
<code>docker-compose logs app</code>	Toon alleen logs van de app-container
<code>docker-compose exec app bash</code>	Open een shell in de app-container
<code>docker-compose exec db mysql -u root -p</code>	Open MySQL-client in de db-container